Celebal Excellence Internship

[Week – 5 Task – Basics of Pyspark]

Title: CEI – week 5 task submission

Name: Anushka Manoj Patil

College name: Dr. D. Y. Patil Pimpri, Pune

Q1: Limitations of MapReduce and why Spark is preferred?

Traditional MapReduce has following limitations:

1. **Disk-based processing**
    - MapReduce writes intermediate results to disk.
    - This causes slow execution.
2. **High latency**
    - Each job starts a new process.
3. **Not suitable for iterative algorithms**
    - Machine learning algorithms require repeated access to data.
4. **Complex programming model**
    - Requires separate Map and Reduce functions.

Spark is preferred because:

- Uses in-memory computation.
- Faster processing.
- Supports iterative algorithms.
- Provides DataFrames, SQL, Streaming, ML libraries.
- Easier programming.

Q2: Explain how Spark uses In-Memory Computing to speed up iterative machine learning algorithms compared to disk-based systems.

Spark keeps intermediate data in RAM instead of repeatedly writing to disk.

Example:

Machine learning algorithms like:

- Logistic Regression
- K-Means
- Neural Networks

perform multiple iterations.

MapReduce:

Read data → Process → Write disk

Read again → Process → Write disk

Spark:

Read data → Store in memory → Multiple iterations

Therefore Spark is much faster.

Q3. Write a code snippet to remove all duplicate rows from a DataFrame based on a specific set of columns: user_id and transaction_date.

```
df = df.dropDuplicates(
["user_id","transaction_date"]
)
```

Q4. Given a DataFrame df_sales, write a query to filter for rows where the region is 'West' and then group by product_category to find the average sale_amount.

Query:

```
from pyspark.sql.functions import col
df=df.withColumn("Sales", col("Sales").cast("double")
from pyspark.sql.functions import avg
df.filter(col("Region")=="West")\
.groupby("Category")\
.agg(avg("Sales").alias("Average_Sales"))\
.show()
```

Observation:

Aggregate functions can be applied on numerical columns only.

And Sales is not a numerical column so we need to convert it to integer.

Hence here we are first converting the type of the sales column from String to double.

Q5: What is the difference between .na.drop() and .na.fill()? Provide a code example of filling null values in a status column with the string 'Unknown'.

na.drop()

Removes rows containing null values.

Example:

```
df.na.drop()
```

na.fill()

Replaces null values.

Example:

```
df.na.fill(
   {"status":"Unknown"}
)
```

Q6.Write a query to find the total count of records for each city in a DataFrame, but only for cities where the count is greater than 100.

Query:

```
from pyspark.sql.functions import count


df.groupBy("city")\
.agg(
   count("*").alias("total")
)\
.filter("total > 100")\
.show()
```

Q7: How does the immutability of Spark DataFrames affect how you perform "data cleaning" steps like dropping columns or renaming them?

Spark DataFrames are **immutable**, which means once a DataFrame is created, it **cannot be modified directly**. Any operation such as dropping a column, renaming a column, filtering rows, or handling null values **creates a new DataFrame** instead of changing the original one. Therefore, during data cleaning, the result of each transformation should be assigned to a new DataFrame or overwrite the existing DataFrame variable.

Example:

```
# Drop a column
df = df.drop("Discount")
```

```
# Rename a column
df = df.withColumnRenamed("Customer Name", "Customer_Name")

# Remove duplicate rows
df = df.dropDuplicates()
```

Explanation:
The original DataFrame remains unchanged until you assign the transformed DataFrame back to a variable (such as df).

Q8. Write a Spark command to filter a dataset for rows where the age is between 18 and 30 (inclusive) and the subscription is 'Premium'.

Query:

```
    from pyspark.sql.functions import col

df.filter(
   (col("age").between(18, 30)) &
   (col("subscription") == "Premium")
).show()
```

Q9: When cleaning a dataset, why is it often better to handle null values before performing mathematical aggregations like sum() or avg()?

Answer:

It is better to handle null values before performing mathematical aggregations because null values can lead to inaccurate or incomplete results. If null values are ignored or left untreated, calculations such as sum(), avg(), min(), and max() may not reflect the true values in the dataset. Replacing or removing null values ensures that aggregations produce meaningful and reliable results.

Example:

```
from pyspark.sql.functions import sum


# Fill null values in Sales with 0

df = df.na.fill({"Sales": 0})


# Calculate total sales

df.agg(sum("Sales").alias("Total_Sales")).show()
```

Q10: Write the code to revise a column named raw_timestamp by casting it to a TimestampType and renaming it to event_time.

```
from pyspark.sql.functions import col
from pyspark.sql.types import TimestampType

df = df.withColumn(
    "event_time",
    col("raw_timestamp").cast(TimestampType())
).drop("raw_timestamp")

df.show()
```

Q11.Explain the "Shuffle" process that occurs during a grouping operation. Why is it considered a wide transformation?

A shuffle is the process in Spark where data is redistributed across different partitions so that records with the same key are brought together for operations like groupBy(), join(), and reduceByKey().

For example, when you perform:

df.groupBy("Category").count().show()

Spark moves all rows belonging to the same Category into the same partition before counting them. This movement of data between partitions is called a shuffle.

It is considered a wide transformation because data moves across multiple partitions and often across different worker nodes in a cluster. Unlike narrow transformations (such as filter() or select()), wide transformations require network communication, making them more expensive in terms of time and resources.

Example:

```
from pyspark.sql.functions import count


df.groupBy("Category") \
  .agg(count("*").alias("Total_Products")) \
  .show()
```

This groupBy() operation triggers a shuffle, as Spark must redistribute rows so that all records with the same Category are processed together.

Q12: Write a code snippet that identifies and removes rows where the email column contains null values OR the username is an empty string.

Query:

```
        from pyspark.sql.functions import col


df = df.filter(
    col("email").isNotNull() &
    (col("username") != "")
)


df.show()Observation:
```

This query first calculates Total count of orders by using aggregate function COUNT() inside the CTE.

Then by using HAVING it groups the orders with having count 1.

Q13: How do you use the .agg() function to calculate multiple statistics at once, such as the min, max, and mean of the price column?

Query:

```
    from pyspark.sql.functions import min, max, avg


df.agg(
   min("price").alias("Minimum_Price"),
   max("price").alias("Maximum_Price"),
   avg("price").alias("Average_Price")
).show()Observation:
```

This query first calculates sum of orders by using aggregate function SUM() inside the CTE.

Then by using the subquery it calculates average of total sales.

And compares total sales with average sales.

Q14.In the context of cleaning a dataset, what is the risk of using inferSchema=True when your source data contains messy or inconsistent date formats?

Using inferSchema=True allows Spark to automatically detect the data type of each column. However, if the dataset contains

inconsistent or messy date formats (such as 11/08/2016, 2016-11-08, and 08-Nov-2016 in the same column), Spark may:

- Infer the wrong data type.
- Treat the entire column as a string.
- Produce null values for invalid date formats after casting.
- Lead to incorrect filtering, sorting, or aggregation.

For reliable processing, it is often better to define the schema manually or explicitly convert date columns using functions like to_date() or to_timestamp().

Q15: Write a final processing pipeline that:

Using Filters out duplicates.

- **Fills null prices with 0.**
- **Groups by store_id to calculate total revenue.**

Answer

```
from pyspark.sql.functions import sum

result = (
   df
   .dropDuplicates()
   .na.fill({"price": 0})
   .groupBy("store_id")
   .agg(sum("price").alias("Total_Revenue"))
)
```

```
result.show()
```